A very similar method to the above is to use `mb_check_encoding()`:

```
$invalidDoubleByte = "\xC2\x28";
$validUTF8 = "\u{141}ukasz";

var_dump(ini_get('mbstring.strict_detection'));
var_dump(mb_check_encoding($validUTF8));
var_dump(mb_check_encoding($validUTF8 . $invalidDoubleByte));

mb_internal_encoding("UTF-16BE"); // previous int. enc. was default UTF-8
var_dump(mb_check_encoding($validUTF8));

Output:
string(1) "0"
bool(true)
bool(false)
bool(false)
```

Even without `strict_detection` (which is disabled – set to `0`), the input string is strictly tested. It is possible to set the desired encoding to test against with an additional argument to `mb_check_encoding()`; we'll see that in a moment. Otherwise the `internal_encoding` setting is used. From the output of the example above, we can deduce the following: the input string is tested for correctness against `internal_encoding` (valid UTF-8 returns true initially, and false when internal encoding is changed to UTF-16BE), and the input string must not have any invalid sequences (adding a broken continuation sequence to the valid UTF-8 string results in a `false` return).

If an encoding type other than `internal_encoding` is required, pass the name as an argument:

```
$validSurrogatePair = "\xD8\x00\xDC\x01"; //LINEAR B SYLLABLE B038 E 𐂁

var_dump(mb_check_encoding($validSurrogatePair));
var_dump(mb_check_encoding($validSurrogatePair, 'UTF-16BE'));

Output:
bool(false)
bool(true)
```